

TECHNICAL REPORT

IoT / EMBEDDED SYSTEMS

Task 3.1: Node-RED Dashboard for Visual Control and Live Telemetry

Daily Technical Report, Day 18 + Day 19

Platform	ESP32-S3-WROOM · Arduino Framework · PubSubClient
Subsystem	Node-RED dashboard: live BME280 telemetry, L298N motor control and true-state feedback
Broker	Self-hosted Mosquitto 2.0.22 inside WSL 2 (Ubuntu), reused from Task 11. No second broker was installed.
Runtime	Node.js v24.19.0 and Node-RED installed on Windows through npm, run from PowerShell on port 1880
Dashboard	node-red-dashboard palette (ui_gauge, ui_chart, ui_text, ui_switch, ui_slider, ui_button)
Clients	Node-RED dashboard (laptop and phone browser), MQTT Explorer, mosquitto_pub / mosquitto_sub, ESP32-S3 firmware
Prepared By	Malak Alshrqawi
Prepared For	IOT Training Program

Confidential, Internal Training Documentation

Contents

1. Daily Overview	4
1.1 Summary	4
1.2 Goal & Submission Format	4
1.3 Tasks Completed	4
2. Part 1: Installing the Node-RED Environment on Windows	5
2.1 Why Node-RED Runs on Windows While the Broker Stays in WSL	5
2.2 Node.js as a Prerequisite	6
2.3 Installing and Verifying Node.js	6
2.4 The PowerShell Execution Policy Problem	6
2.5 Installing Node-RED Globally	7
2.6 Running Node-RED from the Terminal	7
2.7 The Node-RED Editor: Palette, Workspace, Sidebar, Deploy	8
2.8 First Flow: Inject to Debug	8
2.9 Installing the Dashboard Palette	8
3. Connecting Node-RED to the Existing Mosquitto Broker	10
3.1 One Broker, Not Two	10
3.2 Why localhost Works Instead of the WSL IP	10
3.3 Verifying That Mosquitto Is Listening	10
3.4 The First MQTT Flow: MQTT IN to Debug	11
3.5 Adding the Remaining Sensor Topics	12
4. System Schematic: Hardware Wiring and Node-RED Flow	13
4.1 Hardware Wiring and Signal Path	13
4.2 The Node-RED Flow, Node by Node	14
4.3 What the Flow Confirms and What It Reveals	16
5. Topic Model: Command Topics vs Telemetry Topics	17
5.1 Why the Separation Is Necessary	17
5.2 Telemetry Topics (ESP32 to Node-RED)	17
5.3 Command Topics (Node-RED to ESP32)	17
5.4 Complete Topic Tree	18
5.5 Firmware Alignment Note	18
6. Part 2: Dashboard Layout and Live Telemetry	19
6.1 Layout and Grouping	19
6.2 Environmental Telemetry: Gauges and Historical Charts	19
6.3 Value Format: Correct Data, Wrong Presentation	20
6.4 Motor State Telemetry	20
6.5 Motor Direction Telemetry	20
6.6 Motor Speed Telemetry	21

6.7 Why Actual Telemetry Matters: Command State vs Actual State.....	21
7. Connection Status Detection: MQTT Last Will and Testament.....	21
7.1 The Stale Dashboard Problem	21
7.2 LWT Topic and Payloads.....	21
7.3 Configuring the Last Will on the ESP32.....	22
7.4 Publishing the Retained Online State.....	22
7.5 LWT Flow and the Node-RED Connection Indicator	23
7.6 Detection Timing and Keep-Alive	23
8. Part 2: Control Panel, Node-RED to ESP32.....	23
8.1 Motor Power Control	23
8.2 Motor Direction Control	24
8.3 Motor Speed Control.....	24
8.4 Speed Validation and Clamping Inside Node-RED	24
8.5 Slider Publishing Behaviour	25
8.6 Emergency Stop	25
9. Part 3: Requirements Compliance	26
10. End-to-End Verification & Screenshots	26
11. Definition of Done	28
12. Remaining Work and Next Steps	28
13. References.....	28

1. Daily Overview

1.1 Summary

Day 18 and Day 19 close the last gap in the control loop built over the earlier MQTT task. Up to this point the only way to talk to the ESP32 was MQTT Explorer, which is a debugging tool rather than an interface: it shows raw topics and raw payloads and assumes whoever is using it already knows the namespace by heart. These two days replaced it with a Node-RED dashboard running locally on the Windows host against the same Mosquitto broker, so the whole system can now be watched and driven from a browser on a laptop or a phone.

The work was deliberately layered rather than rushed. Node.js was installed and verified first, then a PowerShell execution policy problem that blocked npm was diagnosed and fixed, then Node-RED was installed globally and run from the terminal, then a trivial inject to debug flow proved the editor worked at all, then a single MQTT IN node proved that real ESP32 data could reach Node-RED, and only after that were any gauges created. Each layer was tested on its own, so when something failed there was only ever one new variable to blame.

The dashboard that came out of it is two-directional. In one direction the ESP32 publishes BME280 readings and its own true motor state, and Node-RED renders those as gauges, historical charts and colour-coded indicators. In the other direction the dashboard publishes command payloads on a separate cmd branch of the topic tree, which the ESP32 subscribes to, validates and applies to the L298N driver. The two directions never share a topic, which is what allows the dashboard to display what the hardware is actually doing instead of echoing back what the user just asked for. The third piece is presence detection: an MQTT Last Will and Testament on esp32/status, so that if the ESP32 loses power or drops off the Wi-Fi network, the broker announces it on the device's behalf and the dashboard flips to OFFLINE instead of freezing on stale values.

1.2 Goal & Submission Format

Goal: build on the MQTT system from Task 11 by creating a live Node-RED dashboard that visualises sensor data and controls the motor, so that MQTT Explorer is no longer needed to talk to the ESP32.

Submission Format

GitHub repository link + exported Node-RED flow (flows.json) + dashboard screenshots or screen recording.

1.3 Tasks Completed

- Installed Node.js on Windows and verified it from PowerShell with `node --version`, which returned v24.19.0.
- Diagnosed an npm failure that was not an npm failure: the `npm.ps1` wrapper was blocked by the PowerShell execution policy, and it was fixed for the current user only with `Set-ExecutionPolicy RemoteSigned`.
- Installed Node-RED globally through npm rather than using a hosted service, and confirmed the install with `node-red --version`.
- Ran Node-RED from the terminal so its log stays visible, mirroring the way Mosquitto is run in Task 11, and opened the editor at `http://localhost:1880`.
- Learned the editor layout, palette, workspace, sidebar and Deploy, and ran an inject to debug flow to confirm messages move between nodes.
- Installed the dashboard palette from Manage palette so the `ui_` widget nodes became available.

- Confirmed from WSL that Mosquitto was listening on 0.0.0.0:1883 before pointing anything at it.
- Connected Node-RED to the existing broker over localhost:1883, reusing the Windows to WSL port forwarding rather than the WSL virtual IP, and without starting a second broker.
- Proved the path end to end with a single MQTT IN node on esp32/sensor/temperature wired to a debug node, then added humidity and pressure.
- Built the environmental section: a live gauge and a historical chart for each of temperature, humidity and pressure.
- Fixed a gauge that showed no value by configuring the Value Format field, a presentation problem rather than a data problem.
- Added motor state, motor direction and motor speed indicators driven by the ESP32 telemetry topics, not by the last command sent from the UI.
- Implemented MQTT Last Will and Testament on the ESP32 with a retained online payload, and added an ONLINE / OFFLINE indicator on the dashboard.
- Built the control panel: motor power switch, direction control and a 0 to 100 percent speed slider, all publishing to the cmd branch.
- Laid out an Emergency group that is visually separate from the normal Control group.

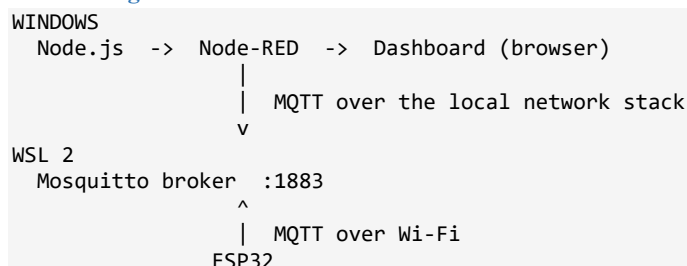
2. Part 1: Installing the Node-RED Environment on Windows

2.1 Why Node-RED Runs on Windows While the Broker Stays in WSL

The task specifies that Node-RED is installed locally on the Windows machine through npm. Mosquitto, from Task 11, lives inside WSL 2. That split is deliberate and it is worth stating plainly, because it is the source of every networking question that follows.

Environment	What runs there	Why
WSL 2 (Ubuntu)	Mosquitto broker on port 1883, Linux development tools, the ESP32 project environment	The broker was set up here in Task 11 and is already reachable by the ESP32 through the Windows firewall rule and the netsh portproxy bridge.
Windows	Node.js, Node-RED, the browser, the dashboard	Required by the task, and it puts the dashboard on the machine the operator actually uses.

The resulting architecture



Running Node-RED from a terminal rather than as a background service follows the same philosophy applied to Mosquitto in Task 11: when the runtime prints its own log to a visible window, every MQTT connection, disconnection, deploy and node error is observable at the moment it happens. Running it locally rather than in

the cloud also keeps the entire loop on the LAN, so a command leaving the dashboard reaches the motor with local network latency and the system keeps working with the internet disconnected.

2.2 Node.js as a Prerequisite

Node-RED is a Node.js application, so Node.js has to exist before Node-RED can be installed. Node.js also ships npm, the package manager, which is the tool used to install Node-RED itself and any additional node packages such as the dashboard palette.

The dependency chain

```
Node.js
| provides the runtime and npm
v
Node-RED
|-- MQTT processing
|-- dashboard logic
+-- web dashboard served at /ui
```

2.3 Installing and Verifying Node.js

Node.js was installed on Windows from the official installer, then verified from PowerShell rather than assumed to be working:

PowerShell, verify Node.js

```
node --version
v24.19.0
```

That output confirms two separate things at once: Node.js is installed, and it is on the PATH so it can be called from any terminal directory. The next check, npm, is where the first real problem appeared.

2.4 The PowerShell Execution Policy Problem

Running the npm version check produced an error that reads like a missing installation but is not one:

PowerShell, the error

```
npm --version

npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because
running scripts is disabled on this system.
```

npm was installed correctly, as part of Node.js. On Windows, npm is invoked through a PowerShell wrapper script, npm.ps1, and PowerShell refuses to run any script file while the execution policy forbids it. The command never reaches npm at all.

What actually happens

```
npm command
v
npm.ps1 (a PowerShell script)
v
PowerShell execution policy
+-- script blocked
```

The fix changes the policy for the current Windows user only, rather than for the whole machine:

PowerShell, fix the execution policy

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

Execution Policy Change
```

[Y] Yes [A] Yes to All [N] No
Y

Part of the command	What it does	Why it was chosen
Set-ExecutionPolicy	Changes the rules PowerShell applies when it is asked to run a script file.	This is the actual blocker, not npm or Node.js.
-Scope CurrentUser	Applies the change only to the current Windows user.	Narrower than a machine-wide change, and it does not need an administrator shell.
-ExecutionPolicy RemoteSigned	Locally created scripts run. Scripts downloaded from the internet must be signed or explicitly unblocked.	Keeps the protection that matters, which is against untrusted downloaded scripts. Unrestricted would have removed that protection for no benefit.

PowerShell reads the policy when the session starts, so the current window has to be closed and a new one opened before the change takes effect. Running the check again then returns a version number in the 10.x range, and at that point Node.js, npm and PowerShell are all in a working state.

2.5 Installing Node-RED Globally

PowerShell, install Node-RED

```
npm install -g --unsafe-perm node-red
```

The -g flag installs Node-RED globally rather than into a single project folder, which is what puts the node-red command on the system path so it can be started from any directory. Without it, the install would be tied to whichever directory it was run in. The install is then confirmed the same way Node.js was:

PowerShell, verify Node-RED

```
node-red --version
```

2.6 Running Node-RED from the Terminal

Terminal, start Node-RED

```
node-red

Welcome to Node-RED
=====
...
Server now running at http://127.0.0.1:1880/
```

The terminal stays open for as long as Node-RED is running, which means the project now has two long-lived terminals side by side, in the same style as Task 11:

Terminal	Environment	Process	Why it stays open
Terminal 1	WSL 2 (Ubuntu)	mosquitto -c mosquitto.conf -v	Verbose broker log: every client connect, subscribe and disconnect is visible as it happens.
Terminal 2	Windows PowerShell	node-red	Runtime log: flow deploys, MQTT connection state and node errors are visible as they happen.

Two URLs matter from here on. The flow editor is at <http://localhost:1880> and the dashboard itself at <http://localhost:1880/ui>, which is the address visible in the browser bar of Figure 6. To reach the dashboard from a phone on the same Wi-Fi network, the same paths are used with the Windows LAN IP in place of localhost.

2.7 The Node-RED Editor: Palette, Workspace, Sidebar, Deploy

Before connecting anything to MQTT, the editor itself has to make sense. It has three areas and one button that matters.

Area	Position	Purpose
Palette	Left	The available node types, grouped into categories such as common, function, network, sequence, parser, storage and dashboard. Nodes are dragged from here onto the canvas.
Workspace	Middle	Where nodes are placed and joined. The line joining two nodes is a wire, and messages travel along it from left to right.
Sidebar	Right	Information, help and the Debug panel, which prints the messages passing through any debug node.
Deploy	Top right	Node-RED does not apply edits as they are made. Changes only take effect when Deploy is pressed.

The working cycle

Create nodes -> Connect nodes -> Configure nodes -> Deploy -> Test

2.8 First Flow: Inject to Debug

The first flow built was deliberately trivial: an inject node wired to a debug node. Pressing the inject button produces a message, and the message appears in the Debug sidebar.

The first flow

```
[ Inject ] -----> [ Debug ]
```

Inject creates a message
Debug displays the message

It looks unrelated to MQTT, and that is the point. It establishes the one concept everything else in Node-RED rests on, which is that nodes generate, modify, process and consume messages, and it proves the editor and the deploy cycle work before any external system is involved. If the inject test had failed, no amount of MQTT configuration would have helped.

2.9 Installing the Dashboard Palette

Node-RED ships the flow editor but not the UI widgets. Gauges, charts, switches, sliders and buttons come from a dashboard package, installed from inside the editor through Menu, Manage palette, Install, then searching for the package by name.

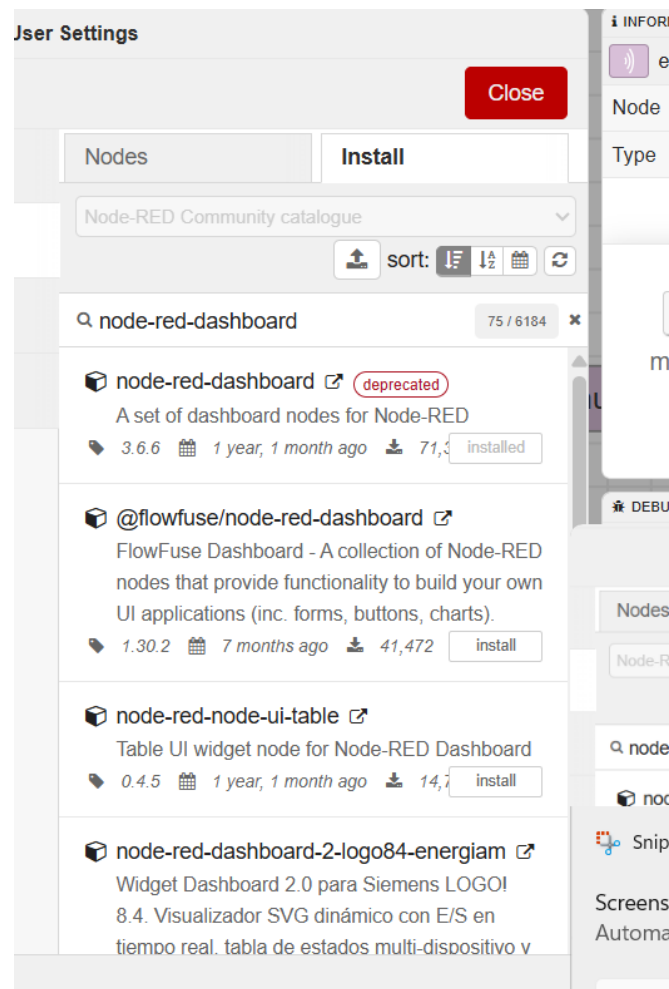


Figure 1. The Manage palette install dialog. `node-red-dashboard` 3.6.6 is marked installed and flagged deprecated, while `@flowfuse/node-red-dashboard` 1.30.2 is offered as a separate install.

This screenshot settles a question worth being precise about in the README. Two packages exist. The original `node-red-dashboard` is the one that is installed in this environment, and it is the one that provides the `ui_gauge`, `ui_chart`, `ui_text`, `ui_switch`, `ui_slider` and `ui_button` node types used throughout the rest of this report, along with the dashboard served at `/ui` that appears in Figures 6 and 7. Its maintainers declared it deprecated on 27 June 2024: it still installs and runs, but it is based on AngularJS v1 and will not receive further development. The successor is FlowFuse Dashboard, published as `@flowfuse/node-red-dashboard` and commonly called Dashboard 2.0, which the same dialog shows as available but not installed.

Note for the README

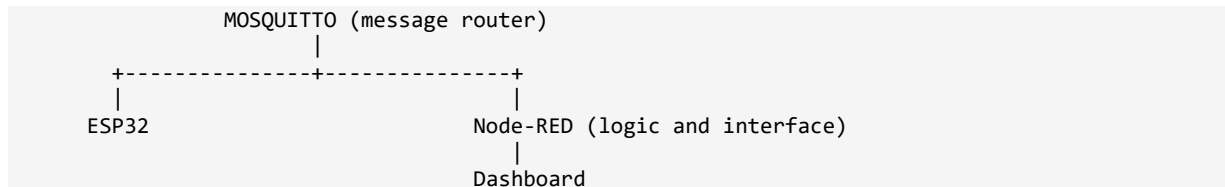
The working notes for these two days mention installing the FlowFuse package, while the palette dialog and the widget types actually in the flow both point to the original `node-red-dashboard`. The README should name the package that is really in use, since a reviewer reproducing the flow from `flows.json` will install whichever one is stated. The topic design, the clamping logic and the LWT mechanism described in this report all sit outside the widget layer, so a later migration to Dashboard 2.0 would only touch the UI nodes.

3. Connecting Node-RED to the Existing Mosquitto Broker

3.1 One Broker, Not Two

The single most important rule at this stage is that Node-RED connects to the broker that already exists. Node-RED is not an MQTT broker and does not contain one. Mosquitto routes messages between clients, and Node-RED is simply another client alongside the ESP32 and MQTT Explorer. Installing a second broker would have produced two isolated message spaces, and the dashboard would have sat there subscribed to topics that nothing was ever going to publish to.

Who does what



3.2 Why localhost Works Instead of the WSL IP

The broker runs inside WSL 2, which sits on its own NAT-isolated virtual subnet, so the obvious assumption is that Node-RED must be pointed at the WSL virtual IP. It was checked:

Inside WSL, find the virtual IP

```
hostname -I
172.29.163.242
```

That address turned out to be unnecessary. Task 11 already configured Windows to forward port 1883 into WSL through a netsh portproxy rule, which is what lets the ESP32 reach the broker at the Windows LAN IP. Node-RED runs on that same Windows host, so it can use the near end of the same forwarding path and connect to localhost.

The path a Node-RED packet takes

```

Node-RED
| localhost:1883
v
Windows port 1883
| netsh portproxy (NAT)
v
WSL port 1883
v
Mosquitto
  
```

Using localhost is also the more robust choice, not just the more convenient one. The WSL virtual IP is assigned by the Hyper-V NAT switch and changes when WSL restarts, so a hardcoded 172.29.x.x address in the broker configuration node would silently break the dashboard on the next reboot. Note the asymmetry that follows: the ESP32 connects to the broker at the Windows LAN IP because it is a separate device on the Wi-Fi network, while Node-RED connects at localhost because it is on the host itself. Both reach the same broker.

3.3 Verifying That Mosquitto Is Listening

Before pointing Node-RED at anything, the broker was confirmed to be up and bound correctly from inside WSL:

WSL, check the listening socket

```
sudo ss -lntp | grep 1883

LISTEN 0 100 0.0.0.0:1883 0.0.0.0:* users:(("mosquitto",pid=5729,fd=4))
LISTEN 0 100 [::]:1883 [::]:* users:(("mosquitto",pid=5729,fd=5))
```

The part that matters is 0.0.0.0:1883. It means Mosquitto is bound to every IPv4 interface rather than only to the loopback address, which is the configuration set in mosquitto.conf during Task 11 and the reason external traffic can reach it at all. Checking this first costs one command and removes an entire class of false leads later: if this line is missing, no amount of Node-RED configuration will help.

3.4 The First MQTT Flow: MQTT IN to Debug

With the broker confirmed, one MQTT IN node was added and pointed at a topic the ESP32 was already publishing, wired to nothing more ambitious than a debug node.

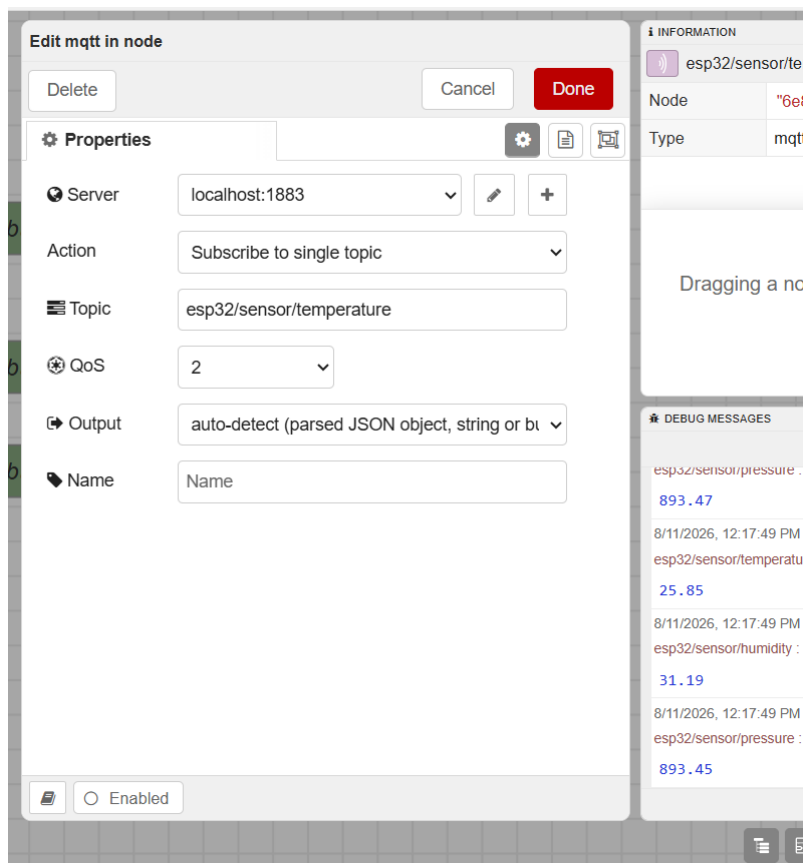


Figure 2. The MQTT IN node configuration: the broker entry reads localhost:1883, the action is a single-topic subscription, and the topic is one the ESP32 already publishes.

Field	Value	Comment
Server	localhost:1883	The existing Mosquitto broker, reached through the Windows to WSL port forwarding. One shared broker configuration node is reused by every MQTT node in the flow.
Action	Subscribe to single topic	One topic per node at this stage. A wildcard subscription would have made the first test harder to read.
Topic	esp32/sensor/temperature	An existing topic. No new topics were invented for Node-RED.

Field	Value	Comment
QoS	2	Requested by the subscriber. Effective delivery is still QoS 0 here, because PubSubClient on the ESP32 only publishes at QoS 0 and the delivered quality of service is the lower of the two.
Output	auto-detect	Parses a JSON object where possible and otherwise passes a string or buffer. The debug output shows msg.payload arriving as a number, which is what the gauge and chart widgets need.

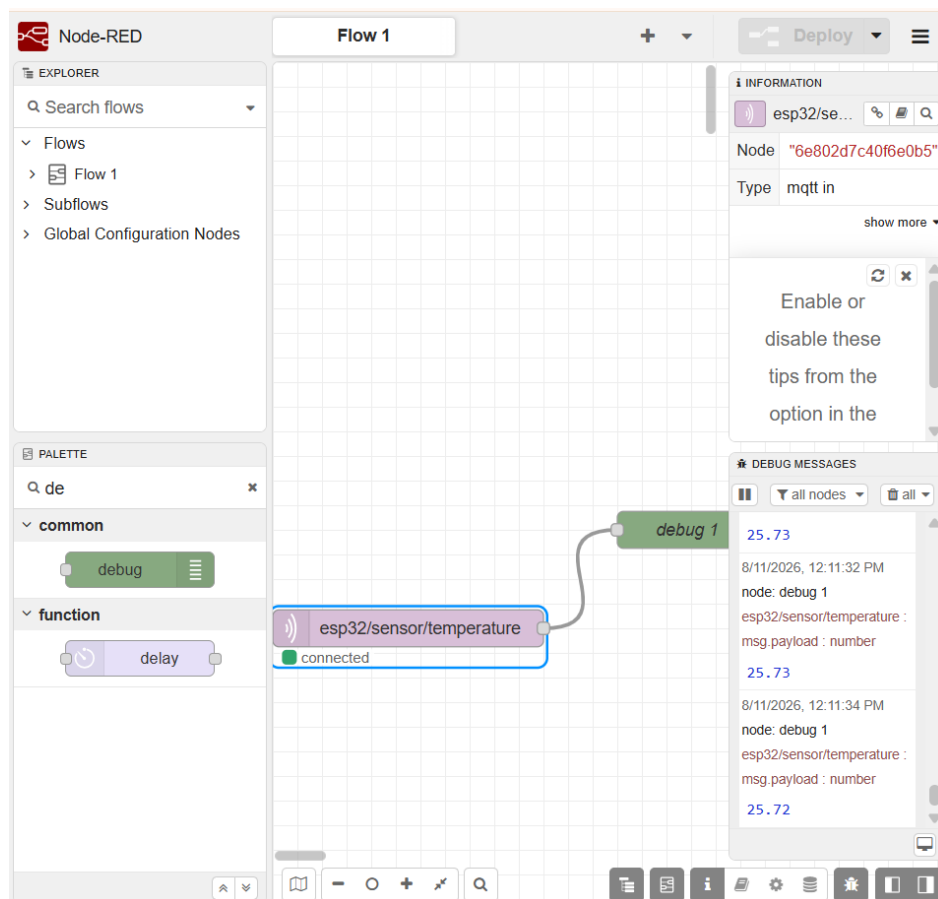


Figure 3. The first successful MQTT test. The MQTT IN node shows connected, and the Debug sidebar prints timestamped temperature payloads arriving from the ESP32 as numbers.

Seeing values in the Debug panel confirmed something a gauge could never have confirmed on its own: that real MQTT data from the ESP32 was reaching Node-RED, rather than a message Node-RED had generated internally. The full path was now proven.

The proven path

```
ESP32 -> Mosquitto (WSL) -> Windows localhost:1883 -> MQTT IN -> Debug
```

3.5 Adding the Remaining Sensor Topics

Once temperature worked, humidity and pressure were added the same way. No new topics were created on the device side, since the ESP32 had been publishing all three since Task 11.

Sensor	MQTT topic	Purpose
Temperature	esp32/sensor/temperature	Current temperature from the BME280
Humidity	esp32/sensor/humidity	Current relative humidity
Pressure	esp32/sensor/pressure	Current atmospheric pressure

With all three arriving in the Debug panel, the layered approach had done its job: the ESP32 was known good from Task 11, the broker was known good, and now the Node-RED link was known good too. Only the dashboard layer remained untested, so any problem from this point on could only be a dashboard problem.

4. System Schematic: Hardware Wiring and Node-RED Flow

Two schematics describe this system, and they meet at the broker. One is the physical wiring that turns an MQTT payload into a spinning shaft, the other is the Node-RED flow that turns a topic into a widget. Neither is complete on its own, so both are reproduced here.

4.1 Hardware Wiring and Signal Path

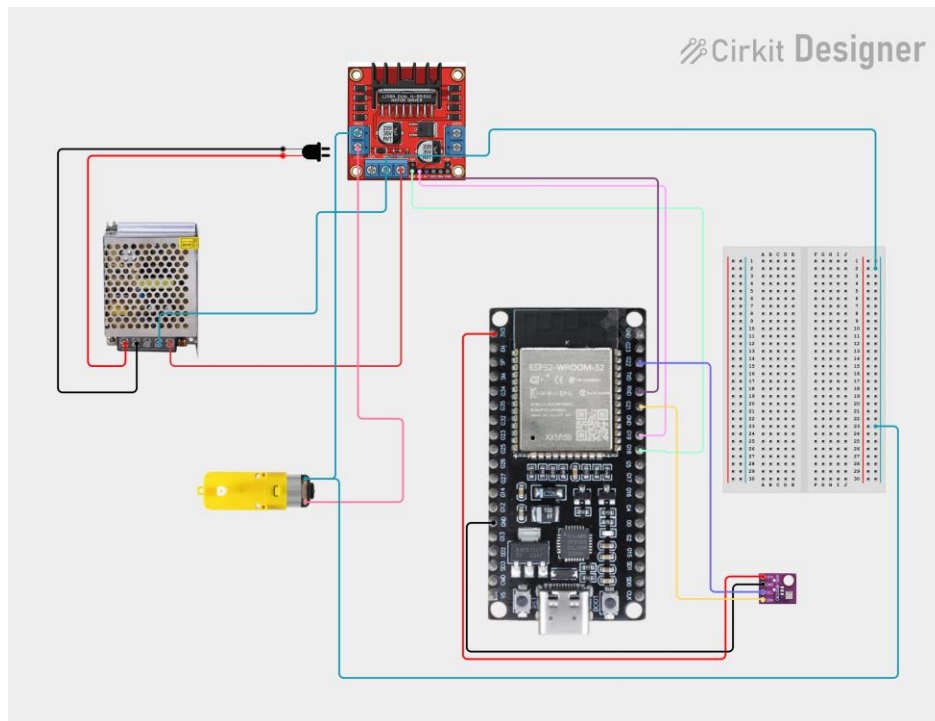


Figure 4. Hardware wiring carried over from the MQTT task: ESP32-WROOM development board, L298N dual H-Bridge driving a DC gear motor, a BME280 module on the I2C bus, and a bench power supply feeding the motor rail, with a breadboard used for distribution and test points.

The wiring separates two power domains and one signal domain. The bench supply feeds the motor rail on the L298N, the ESP32 runs from its own USB-C supply, and the only thing crossing between them at logic level is three control lines and a shared ground. That shared ground is not optional detail, it is the reference the H-Bridge uses to read the direction pins, and without it the driver sees floating inputs and behaves unpredictably even though the wiring looks complete.

Connection	From	To	Purpose
Motor supply	Bench power supply output	L298N motor voltage and ground terminals	Supplies the motor rail. Motor current never passes through the ESP32.
Motor output	L298N OUT1 and OUT2	DC gear motor terminals	The switched H-Bridge output. Swapping these two wires reverses what forward means.
Direction lines	Two ESP32 GPIO pins	L298N IN1 and IN2	Set rotation direction. Both LOW isolates the motor, which is what stop() does.
Speed line	One ESP32 PWM-capable GPIO	L298N ENA	Carries the analogWrite PWM duty, 0 to 255, that sets speed. The onboard ENA jumper must be removed for this to have any effect.
Sensor bus	ESP32 SDA and SCL pins	BME280 module	I2C telemetry bus. The module answers on address 0x76 or 0x77, and the firmware probes both.
Sensor power	ESP32 3V3 and GND	BME280 VCC and GND	The sensor draws microamps, so it runs directly off the board regulator.
Common ground	Bench supply ground	L298N ground and ESP32 GND	Ties both supplies to one reference so the logic levels on IN1, IN2 and ENA are meaningful.

One note on scope. The architecture sketches in the working notes list an IR sensor alongside the BME280 and the motor on the ESP32 block. Nothing in Task 3.1 subscribes to or displays an IR value, so it is out of scope for this dashboard and no topic was reserved for it.

4.2 The Node-RED Flow, Node by Node

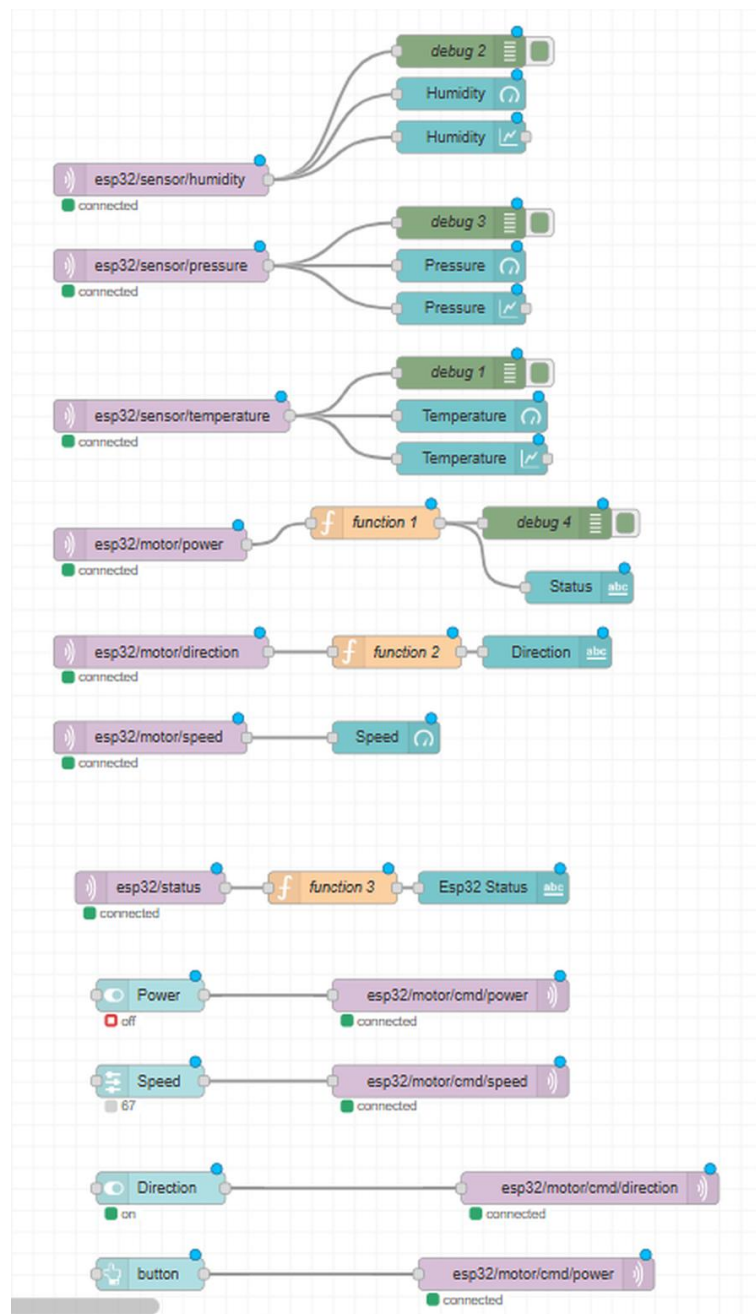


Figure 5. The complete Flow 1 canvas: sensor telemetry chains at the top, motor telemetry and device presence in the middle, and the control widgets publishing to the cmd branch at the bottom. Every MQTT node reads connected, confirming one live session against the shared broker configuration.

The canvas is organised in horizontal bands that mirror the direction of data. Everything above the midpoint is an MQTT IN node feeding widgets, everything below it is a widget feeding an MQTT OUT node. Reading it top to bottom is the fastest way to check the rule that governs this whole task: no indicator is ever wired to a control, and no control is ever wired to a telemetry topic.

Chain	Nodes in order	What it does
Humidity	MQTT IN esp32/sensor/humidity, then debug 2, Humidity gauge, Humidity chart	One subscription fanned out to a debug node and two widgets in parallel.
Pressure	MQTT IN esp32/sensor/pressure, then debug 3, Pressure gauge, Pressure chart	Same pattern, live value plus trend.
Temperature	MQTT IN esp32/sensor/temperature, then debug 1, Temperature gauge, Temperature chart	Same pattern, live value plus trend.
Motor state	MQTT IN esp32/motor/power, then function 1, then debug 4 and the Status text widget	The function node maps running and stopped to a label and a colour, giving the green or red indicator.
Motor direction	MQTT IN esp32/motor/direction, then function 2, then the Direction text widget	The function node translates backward into REVERSE for display while the payload stays untouched.
Motor speed	MQTT IN esp32/motor/speed, then the Speed gauge	Straight to the gauge, since the ESP32 already publishes a 0 to 100 percentage.
Device presence	MQTT IN esp32/status, then function 3, then the Esp32 Status text widget	Maps online and offline to the green or red presence indicator, fed by the retained payload and the Last Will.
Power control	Power switch, then MQTT OUT esp32/motor/cmd/power	Publishes the ON and OFF setpoint.
Speed control	Speed slider, then MQTT OUT esp32/motor/cmd/speed	Publishes the percentage setpoint. The slider reads 67 in the screenshot.
Direction control	Direction switch, then MQTT OUT esp32/motor/cmd/direction	Publishes forward or backward.
Emergency button	button widget, then MQTT OUT esp32/motor/cmd/power	Publishes on the same command topic as the power switch.

4.3 What the Flow Confirms and What It Reveals

Three things are confirmed by the canvas. Every MQTT node carries a green connected marker, so all eleven of them share one working session against the broker configuration described in Section 3.4. Every indicator is fed from a telemetry topic and none of them is fed from a control widget, which is the requirement in Section 6.7 satisfied structurally rather than by convention. And the presence chain on esp32/status is present and connected, so the Last Will path described in Section 7 is wired end to end.

The canvas also reveals two gaps between the design described in this report and the flow as deployed, both worth fixing before flows.json is exported:

- The speed slider wires directly into the MQTT OUT node with no function node between them. The clamping logic set out in Section 8.4 is therefore not present in the deployed flow, and the only protection against an out of range speed is the constrain() call in the firmware. The task asks explicitly for Node-RED itself to be incapable of publishing an invalid value, so the function node has to be inserted into that wire.
- The emergency button publishes to esp32/motor/cmd/power, the same topic the power switch uses, and still carries the default button label. It therefore behaves as a second OFF rather than an independent stop: it

does not zero the speed setpoint, and it does not reset the switch and slider positions, so the requested state and the actual state can drift apart after it is pressed.

Two smaller observations, neither of which is a requirement. The motor speed chain has a gauge but no historical chart, which is consistent with the task, since only the three environmental values need a graph. And the four debug nodes are development aids: they cost nothing functionally and they made the layered testing described in Section 3 possible, but disabling them before the final export keeps the runtime log readable and makes the submitted flow easier for a reviewer to follow.

5. Topic Model: Command Topics vs Telemetry Topics

5.1 Why the Separation Is Necessary

A command says what the user wants. Telemetry says what the device is doing. Publishing both on the same topic collapses that distinction and produces two concrete failures. First, the dashboard would show the requested state even when the ESP32 is offline and physically cannot act on it, which is exactly the behaviour the task forbids. Second, because the ESP32 subscribes with the wildcard `esp32/motor/#`, any status it publishes under that same branch would be delivered straight back to its own callback, so the device would end up parsing its own telemetry as if it were an incoming command.

The namespace therefore keeps device-to-broker traffic on the plain motor branch and broker-to-device traffic on a dedicated `cmd` sub-branch.

5.2 Telemetry Topics (ESP32 to Node-RED)

Topic	Payload	Meaning	Dashboard widget
<code>esp32/sensor/temperature</code>	float, °C	BME280 temperature reading	Gauge 0 to 50 plus line chart
<code>esp32/sensor/humidity</code>	float, %RH	BME280 relative humidity	Gauge 0 to 100 plus line chart
<code>esp32/sensor/pressure</code>	float, hPa	BME280 barometric pressure	Gauge 0 to 1000 plus line chart
<code>esp32/motor/power</code>	running stopped	Actual motor state derived from the live driver output	Colour-coded state text, green or red
<code>esp32/motor/direction</code>	forward backward stopped	Actual direction currently applied to the H-Bridge	Direction text, presented as FORWARD / REVERSE / STOPPED
<code>esp32/motor/speed</code>	integer 0 to 100	Actual speed converted back from raw PWM	Speed gauge 0 to 100 percent
<code>esp32/status</code>	online offline	Connection presence, retained, offline published by the broker as the Last Will	ONLINE / OFFLINE indicator

5.3 Command Topics (Node-RED to ESP32)

Topic	Payload	Meaning	Dashboard widget
esp32/motor/cmd/power	1 or 0 (also on / off accepted)	Motor ON / OFF setpoint	ui_switch, payload type number
esp32/motor/cmd/direction	forward backward	Direction setpoint	Direction toggle, presented as Forward / Reverse
esp32/motor/cmd/speed	integer, clamped 0 to 100	Speed setpoint as a percentage	ui_slider plus clamping function node

5.4 Complete Topic Tree

MQTT namespace after Task 3.1

```

esp32/
|-- sensor/
|   |-- temperature    (telemetry, float)
|   |-- humidity       (telemetry, float)
|   +-- pressure       (telemetry, float)
|-- motor/
|   |-- power          (telemetry, running | stopped)
|   |-- direction      (telemetry, forward | backward | stopped)
|   |-- speed          (telemetry, 0 to 100)
|   +-- cmd/
|       |-- power      (command, 1 | 0)
|       |-- direction  (command, forward | backward)
|       +-- speed      (command, 0 to 100)
+-- status             (presence, online | offline, retained + LWT)

```

The structure is hierarchical rather than flat, which is what makes the wildcard subscription `esp32/motor/#` workable on the device side: any new command sub-topic added later is picked up automatically without a firmware change, while each telemetry topic stays individually addressable so MQTT Explorer, Node-RED and any other subscriber can watch a single value change on its own.

5.5 Firmware Alignment Note

Moving the commands into the `cmd` branch is a change on both sides, not only in Node-RED. Three points need to hold true in the firmware for the split to work end to end, and they are worth checking against the current sketch before the flow is demonstrated:

- The topic comparisons inside `mqttCallback` must match the new command topics, that is `esp32/motor/cmd/power`, `esp32/motor/cmd/direction` and `esp32/motor/cmd/speed`. If they still compare against `esp32/motor/power`, commands published by the dashboard will arrive but never match, and the motor will not respond.
- The wildcard subscription `esp32/motor/#` already covers the `cmd` sub-branch, so no additional subscribe call is needed.
- The status publisher writes the running or stopped string to `esp32/motor/power`, which is the topic the dashboard state indicator reads. In the earlier firmware that string was published to `esp32/motor/state`, so whichever topic is kept, the Node-RED MQTT IN node and the publish call have to agree.

6. Part 2: Dashboard Layout and Live Telemetry

6.1 Layout and Grouping

Widgets are grouped by function rather than dropped onto one page in the order they were created. The dashboard tab, Environmental Monitor, contains an environmental group per measurement, a motor status group, a device presence group and, on the bottom row, the control groups. Grouping this way is what keeps the layout readable when the browser is narrow: the dashboard reflows groups into a single column on a phone, so a group that mixes unrelated widgets becomes hard to read on a small screen, while a group that holds one measurement stays coherent.

Group	Contents	Direction
Pressure / Temperature / Humidity	Live gauge and historical line chart for each measurement	ESP32 to dashboard
Motor status	Direction text, colour-coded state text, speed gauge	ESP32 to dashboard
Esp32	ONLINE / OFFLINE presence indicator driven by esp32/status	Broker and ESP32 to dashboard
Control	Speed slider, direction toggle, power toggle	Dashboard to ESP32
Emergency	Emergency stop button, kept visually separate from the normal power toggle	Dashboard to ESP32

6.2 Environmental Telemetry: Gauges and Historical Charts

Each of the three BME280 values is subscribed once and then wired to two widgets in parallel, a gauge for the instantaneous value and a chart for the trend. This is one of the advantages of a flow-based runtime: one message can be sent down several paths, so there is no need for the gauge, the chart and the debug node to each hold their own MQTT subscription.

Telemetry flow, one per measurement

```
[ MQTT IN: esp32/sensor/temperature ]
      |
      +-----+-----+-----+
      v         v         v
[ ui_gauge ] [ ui_chart ] [ debug ]
```

Widget	Configuration	Reasoning
Temperature gauge	Range 0 to 50, unit °C	Covers indoor and lab ambient range with usable needle resolution.
Humidity gauge	Range 0 to 100, unit %	Relative humidity is a percentage by definition.
Pressure gauge	Range 0 to 1000, unit hPa	Wide range. At roughly 894 hPa in Amman the needle sits near full scale, so a narrower range such as 850 to 950 would give far better visual resolution.
Line charts	X-axis window of about 35 minutes, one series per chart	Satisfies the requirement for at least a few minutes of history.

The gauge answers what the value is right now. The chart answers whether it is rising, falling or steady, which the gauge cannot show at all. Together they are worth more than either alone, and the task asks for both.

Chart persistence deserves its own note. The requirement is that history is not lost on a browser refresh. The chart node keeps its series in the Node-RED runtime, not in the browser, so a refresh reloads the stored buffer as long as the chart's time window or point limit has not expired and the flow has not been redeployed. A full deploy clears chart state, which is expected behaviour rather than a fault. Seeing a line on the graph does not by itself prove the requirement is met, so this is tested deliberately as described in Section 12.

6.3 Value Format: Correct Data, Wrong Presentation

The first gauge appeared on the dashboard without showing its value, even though the MQTT connection was working and the debug node was printing numbers. The cause was the gauge's Value Format field, which had not been configured. Setting it to interpolate the incoming value and append the unit fixed the display:

Gauge value format

```
{{value}} °C

payload received: 25.4
gauge displays: 25.4 °C
```

The same pattern was applied to humidity with a percent unit and pressure with hPa. The lesson is worth recording because it will come up again: MQTT data can be entirely correct while the dashboard presentation is misconfigured. When a widget looks wrong, the debug node answers the question of which layer to fix, and in this case it pointed away from the network and towards the widget.

6.4 Motor State Telemetry

The state indicator subscribes to esp32/motor/power, where the ESP32 publishes the string running or stopped. A small function node maps the payload to a colour and a label so the widget renders a green marker with RUNNING or a red marker with STOPPED, which satisfies the colour-coding requirement. The value is a report from the driver, not a mirror of the power switch.

Motor state flow

```
[ MQTT IN: esp32/motor/power ]
    |
    v
[ Function: map to label + colour ]
    |
    v
[ ui_text: green RUNNING / red STOPPED ]
```

6.5 Motor Direction Telemetry

The direction indicator subscribes to esp32/motor/direction, which carries forward, backward or stopped. The dashboard presents these in operator language, so backward is displayed as REVERSE, while the MQTT payload itself is left untouched. Presentation is changed at the UI layer only, which keeps the wire format stable for any other subscriber and avoids a second vocabulary inside the protocol.

MQTT payload	Dashboard display
forward	FORWARD
backward	REVERSE
stopped	STOPPED

6.6 Motor Speed Telemetry

Speed is reported on esp32/motor/speed as a percentage. The firmware reads the raw PWM value back out of the L298N instance and converts it before publishing, so the number on the gauge is derived from the driver output rather than from the slider position:

ESP32 side, PWM converted back to a percentage

```
speed = motor.getSpeed();           // raw PWM, 0 to 255
speed = map(speed, 0, 255, 0, 100); // back to a percentage
client.publish("esp32/motor/speed", String(speed).c_str());
```

The gauge is configured with a minimum of 0, a maximum of 100 and the percent unit. One consequence of the double conversion is worth stating plainly: the setpoint goes through `map(percent, 0, 100, 0, 255)` on the way in and `map(pwm, 0, 255, 0, 100)` on the way back, and both use integer arithmetic, so the reported value can land a point or two away from the value that was requested. That is not a bug in the dashboard, it is the round trip through an 8-bit PWM register being reported honestly.

6.7 Why Actual Telemetry Matters: Command State vs Actual State

The task requires the dashboard to reflect real device state rather than echo the last command. The two architectures differ in one step, and that step is the whole point of the exercise:

Correct path, the dashboard reports reality

```
User presses ON
-> Node-RED publishes 1 to esp32/motor/cmd/power
-> ESP32 receives and validates the command
-> Motor hardware is driven through the L298N
-> ESP32 reads the driver's live state
-> publishMotorStatus() publishes the true state
-> Node-RED receives the telemetry
-> Dashboard shows RUNNING
```

Wrong path, the dashboard assumes success

```
User presses ON
-> Dashboard immediately shows RUNNING
```

The second version is wrong because it assumes the motor responded. If the ESP32 is offline, if Wi-Fi has dropped, or if the failsafe in `applyMotorHardware` has forced a stop, the motor is not running and the dashboard would be lying to the operator. This is why the power switch and the state indicator are two separate widgets on the dashboard: the switch expresses the requested state, and the indicator expresses the reported state. Figure 7 shows them agreeing, which is the normal case, but the value of the design is that they are free to disagree when something has gone wrong.

7. Connection Status Detection: MQTT Last Will and Testament

7.1 The Stale Dashboard Problem

MQTT is a session protocol over TCP, and a device that is unplugged does not get the chance to announce anything. Without a presence mechanism the dashboard simply stops receiving updates and keeps rendering the last values it saw, so a screen reading RUNNING, 70 percent, ONLINE can persist long after the ESP32 has lost power. A frozen dashboard is more dangerous than an obviously broken one, because it looks correct.

7.2 LWT Topic and Payloads

A dedicated presence topic was created outside the sensor and motor branches, since it describes the device itself rather than any one subsystem:

Topic	Payload	Published by	Meaning
esp32/status	online	The ESP32, immediately after a successful connect, retained	The device is connected
esp32/status	offline	The broker, on the device's behalf, as the Last Will	The device disconnected unexpectedly

7.3 Configuring the Last Will on the ESP32

The Last Will is registered inside the CONNECT packet, which means it must be declared at connection time and cannot be added later. The plain connect call was therefore replaced with the five argument form:

Before

```
client.connect("ESP32Client");
```

After, with the Last Will registered

```
client.connect(
  "ESP32Client",    // client ID
  "esp32/status",   // will topic
  1,                // will QoS
  true,             // will retain
  "offline"         // will message
);
```

Parameter	Value	Why this value
Client ID	ESP32Client	Identifies the session on the broker. It must not collide with the Node-RED client ID.
Will topic	esp32/status	The presence topic the dashboard indicator subscribes to.
Will QoS	1	At least once delivery for the announcement, so a single dropped packet does not leave the dashboard thinking the device is still online.
Will retain	true	The broker stores offline as the retained value on that topic, so a browser or a Node-RED restart that subscribes later is told the device is down straight away.
Will message	offline	The payload the dashboard maps to the red OFFLINE indicator.

The broker publishes this message when the client goes away without a clean DISCONNECT packet, which covers the cases that matter here: the ESP32 losing power, the Wi-Fi link dropping, a network error, or the keep-alive period elapsing with no traffic from the device. A deliberate, clean disconnect does not trigger the Will, which is the intended behaviour.

7.4 Publishing the Retained Online State

The Will covers the failure case. The healthy case has to be published by the device itself, right after the connection succeeds:

Publishing the online state, retained

```
client.publish("esp32/status", "online", true);
```

The third argument sets the retain flag. Retention is what makes the indicator correct on a late subscriber: Mosquitto keeps the most recent payload on esp32/status, so when Node-RED is restarted or the dashboard is

opened in a new browser tab, the indicator is populated immediately from the retained value instead of sitting blank until the next status change. Since both the online publish and the offline Will are retained, the topic always holds the current answer rather than the last event.

7.5 LWT Flow and the Node-RED Connection Indicator

Healthy connection

```
ESP32 connects
-> publishes "online" (retained)
-> Mosquitto retains "online"
-> Node-RED MQTT IN: esp32/status
-> Dashboard shows green ONLINE
```

Unexpected disconnection

```
ESP32 drops (power loss, Wi-Fi loss, keep-alive timeout)
-> Mosquitto detects the broken session
-> Mosquitto publishes the Last Will "offline" (retained)
-> Node-RED MQTT IN: esp32/status
-> Dashboard shows red OFFLINE
```

On the Node-RED side this is a single MQTT IN node on esp32/status feeding a function node that maps the payload to a label and a colour, then a text widget in the Esp32 group. It is the cheapest chain in the flow and the most valuable, because it is the one that tells the operator whether anything else on the screen can be trusted.

7.6 Detection Timing and Keep-Alive

The requirement is that the dashboard shows the offline status within a few seconds. How fast that actually happens is governed by the MQTT keep-alive, not by Node-RED. If the ESP32 is unplugged, the broker only learns about it when the TCP session breaks or when the keep-alive interval elapses without a packet from the client. PubSubClient sets the keep-alive to 15 seconds by default, and the broker allows one and a half times that interval before declaring the client dead, so a silent power loss can take around 20 to 25 seconds to surface. Calling `client.setKeepAlive()` with a shorter interval, for example 5 seconds, before connecting brings detection down to the few seconds the task asks for, at the cost of slightly more PINGREQ traffic. This is the one timing detail that should be measured with a stopwatch during the disconnect test rather than assumed.

8. Part 2: Control Panel, Node-RED to ESP32

With telemetry in place, the flow was extended in the opposite direction so the dashboard can drive the hardware. All three controls publish to the cmd branch and none of them writes to a telemetry topic.

8.1 Motor Power Control

A `ui_switch` labelled Motor Power sends 1 when it is on and 0 when it is off, with the payload type set to number so the widget does not emit the JavaScript booleans true and false, which the firmware's string comparison would not match.

Power control flow

```
[ ui_switch: Motor Power ]
  | 1 / 0
  v
[ MQTT OUT: esp32/motor/cmd/power ]
```

On the device side the payload is parsed permissively, so the same topic still works from MQTT Explorer or the command line where a human is more likely to type on than 1:

ESP32 side, power parsing

```
targetPower = (message == "1" || message == "on" || message == "ON");
```

8.2 Motor Direction Control

Direction is exposed as a two-state control. The dashboard presents Forward and Reverse, while the payloads published on `esp32/motor/cmd/direction` remain forward and backward, matching the strings the firmware compares against. The firmware accepts nothing else, so a malformed direction payload leaves the previous setpoint untouched rather than putting the H-Bridge into an undefined state.

Direction control flow

```
[ Direction control: Forward / Reverse ]
    | forward / backward
    v
[ MQTT OUT: esp32/motor/cmd/direction ]

ESP32 applies the setpoint, then reports the
resulting direction on esp32/motor/direction
```

8.3 Motor Speed Control

Speed uses a `ui_slider` with a minimum of 0, a maximum of 100 and a step of 1, publishing a percentage that the firmware converts to PWM. The slider value should pass through a validation node before it reaches the MQTT OUT node, never directly.

Speed control flow

```
[ ui_slider: 0 to 100 ]
    |
    v
[ Function: clamp and validate ]
    |
    v
[ MQTT OUT: esp32/motor/cmd/speed ]
```

8.4 Speed Validation and Clamping Inside Node-RED

The firmware already clamps every incoming speed value with `constrain(SpeedValue, 0, 100)`, and that stays in place as the last line of defence. The task nevertheless requires Node-RED itself to be incapable of publishing an invalid value, and the reasoning behind that requirement is sound: the ESP32 is not the only possible consumer of these topics, and a UI that emits garbage and relies on the device to clean it up is a UI that will eventually be pointed at a device that does not.

Function node, clamping logic

```
let speed = Number(msg.payload);

if (!Number.isFinite(speed)) {
    speed = 0;
}

speed = Math.max(0, Math.min(100, speed));

msg.payload = speed;
return msg;
```

`Number()` rather than `parseInt()` is used deliberately, so that a payload such as `50abc` becomes `NaN` and is caught rather than being silently truncated to 50. The guarantee this produces is that every message leaving Node-RED on the speed command topic satisfies $0 \leq \text{speed} \leq 100$.

One caveat, stated here rather than buried: in the flow as captured in Figure 5 the slider is wired straight to the MQTT OUT node, so this function node is the required design but is not yet in the deployed path. Until it is inserted, the only clamp in the system is the constrain() call on the ESP32, which protects the motor but does not satisfy the requirement that Node-RED cannot publish an invalid value.

Input from slider or debug node	Published value
-20	0
0	0
50	50
100	100
150	100
Invalid or non-numeric value	0

The test for this is to bypass the slider entirely and wire an inject node with a payload of 150 and another with - 20 directly into the function node, then watch the published payload in MQTT Explorer or a debug node. Testing through the slider alone proves very little, because the widget will not let its own value leave the configured range in the first place.

8.5 Slider Publishing Behaviour

The task allows publishing on drag or on release, provided the choice is documented. This dashboard publishes on release. Dragging the slider from 0 to 75 therefore produces one MQTT message carrying 75, rather than one message for every intermediate position the handle passes through.

Publish on release

```

Drag slider 0 -> 10 -> 20 -> ... -> 75
Release at 75
  -> validate
  -> publish 75 to esp32/motor/cmd/speed

```

The reason is bandwidth and hardware behaviour rather than aesthetics. Continuous publishing would send dozens of messages per drag, each one triggering a callback, a hardware write and a status publish on the ESP32, which floods the broker log and makes the PWM output chase the handle. The trade-off is a small loss of live feel while dragging, which does not matter for a speed setpoint.

8.6 Emergency Stop

An Emergency group sits beside the Control group, deliberately separated so that the stop is never confused with the ordinary power toggle and stays visible without scrolling. The button is wired, as Figure 5 shows, but it publishes to esp32/motor/cmd/power, the same topic the power switch uses, and it still carries the default button label. In its current form it is a second OFF rather than an independent emergency stop, so this is the one control from the task specification that is not finished, and it is carried into Section 12 as remaining work.

The intended behaviour, so the wiring is unambiguous when it is completed:

- The button publishes a stop command in its own right rather than flipping the power switch, so it works regardless of the state of the other widgets.
- It should force the power command off and the speed setpoint to 0 in the same action, so that a subsequent power-on cannot restart the motor at the previous speed unexpectedly.

- It should also reset the dashboard's own switch and slider positions, so that requested state and actual state stay consistent after the stop.
- It stays visible and reachable on both the laptop and the phone layout, because a stop control that requires scrolling is not a stop control.

9. Part 3: Requirements Compliance

Requirement	How it is satisfied	Status
Node-RED installed locally via npm and run from the terminal	Installed globally with <code>npm install -g --unsafe-perm node-red</code> on Windows, started with <code>node-red</code> in PowerShell, log visible throughout.	Met
Connected to the existing Mosquitto broker, no second broker	One shared broker configuration node at <code>localhost:1883</code> , reusing the Task 11 port forwarding into WSL.	Met
Dashboard reflects real device state, not the last command	Every status widget is fed from a telemetry topic that the ESP32 publishes after reading the L298N driver state. The command topics live on a separate <code>cmd</code> branch and are never wired into an indicator.	Met
Retained messages or LWT so Node-RED knows the ESP32 is offline	Last Will registered in the CONNECT packet on <code>esp32/status</code> with QoS 1 and retain true, plus a retained online publish after each successful connect.	Met
Historical graphs persist across a short session	Chart series are buffered in the Node-RED runtime with a time window of roughly 35 minutes, so a browser refresh redraws the stored history.	To be verified deliberately
Slider input cannot send an invalid value	The clamping function node using <code>Number()</code> , a finite check and <code>Math.max / Math.min</code> is specified in Section 8.4, but the deployed flow wires the slider straight to the MQTT OUT node. The <code>constrain()</code> clamp in the firmware still protects the motor.	Outstanding in Node-RED
Layout usable on laptop and phone browsers	Widgets are organised into functional groups so the dashboard reflows into a single readable column on a narrow viewport.	To be verified on a phone
Emergency stop clearly separate from the normal OFF and always visible	A dedicated Emergency group is placed next to the Control group and the button is wired, but it publishes to the same command topic as the power switch, so it duplicates OFF rather than acting independently.	Outstanding

10. End-to-End Verification & Screenshots

With Mosquitto running verbosely in its WSL terminal, Node-RED running in its PowerShell terminal and the ESP32 connected over Wi-Fi, the dashboard was exercised in both directions. The two figures below are the same dashboard in two different hardware states, which is the clearest evidence that the indicators track the motor rather than the widgets.

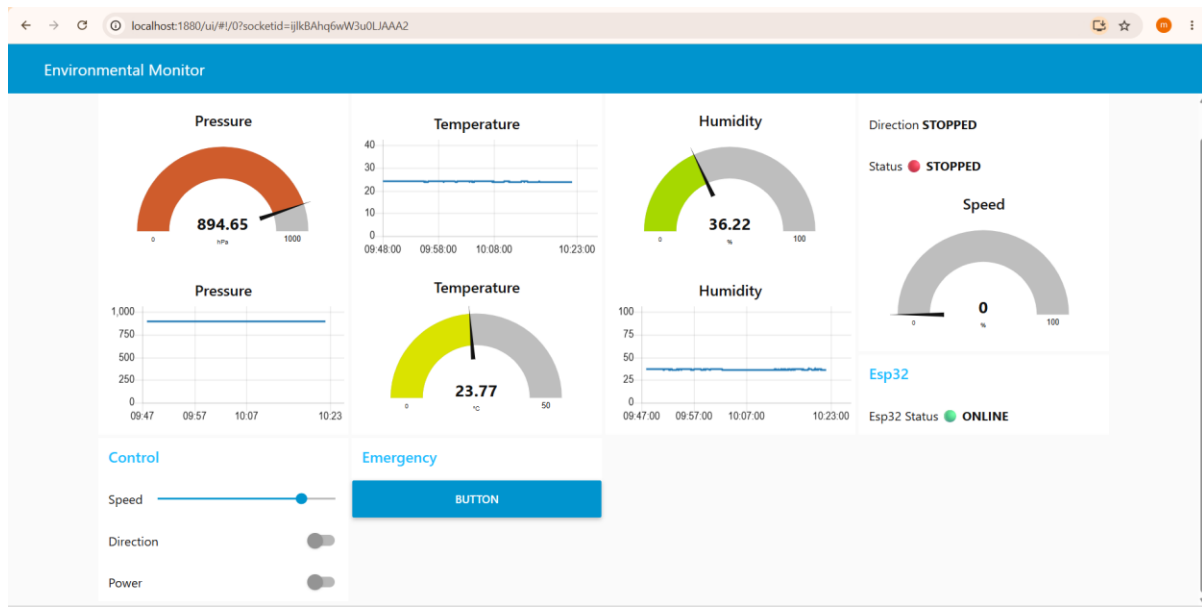


Figure 6. Dashboard at `http://localhost:1880/ui` with the motor powered off. Direction reads STOPPED, the state indicator is red STOPPED and the speed gauge reads 0, while the environmental gauges and charts keep updating and the presence indicator shows a green ONLINE from the retained `esp32/status` payload.

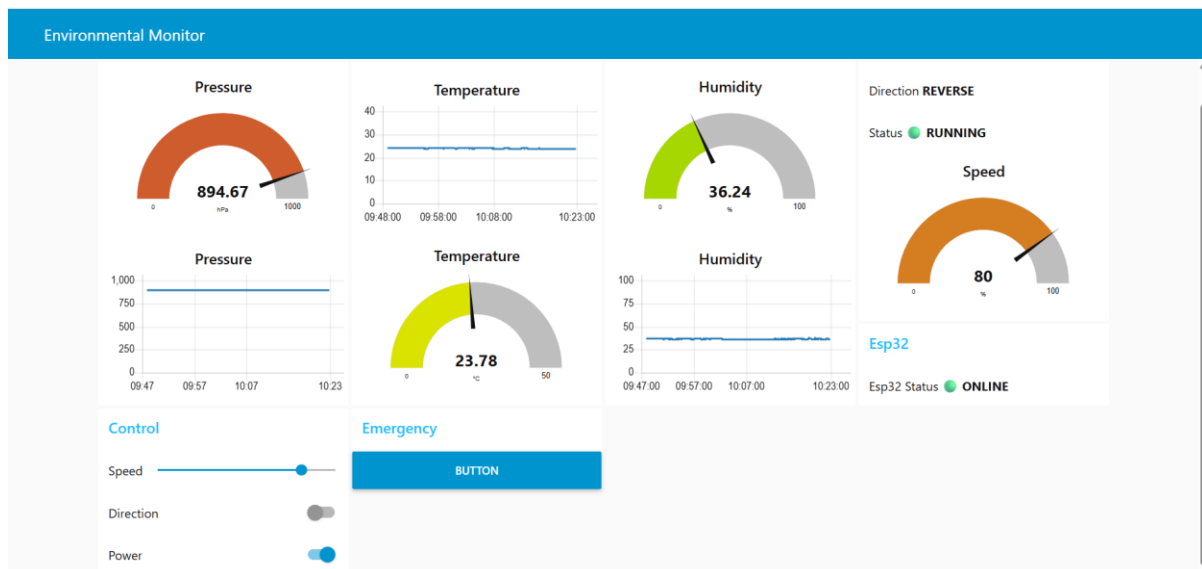


Figure 7. The same dashboard with the motor running in reverse. Direction reads REVERSE, the state indicator is green RUNNING and the speed gauge shows the percentage reported back by the ESP32 after the PWM round trip, not the raw slider position.

Observed behaviour during testing matched the design intent. Publishing an off command, cutting Wi-Fi or stopping the broker caused the motor to stop and the state topic to report stopped on the very next status publish, so the dashboard never stayed on a stale running value. Restoring Wi-Fi or restarting the broker produced an automatic reconnection, a re-subscription and a fresh status publish without an ESP32 reset, and the retained `esp32/status` payload brought the presence indicator back to ONLINE on its own.

Two checks are still worth running with the same rigour before submission: pulling the ESP32's power and timing how long the indicator takes to flip to OFFLINE, as discussed in Section 7.6, and refreshing the browser mid-session to confirm the charts redraw their history rather than starting from an empty axis.

11. Definition of Done

Criterion	Status
Node-RED running from terminal, connected to the local Mosquitto broker	Met, single shared broker config node at localhost:1883
Dashboard shows live temperature, humidity and pressure as both gauges and graphs	Met, see Figures 6 and 7
Motor state, direction and speed shown live and reflecting real hardware behaviour	Met, driven by publishMotorStatus() telemetry
ON/OFF, direction and speed controls actually control the motor	Met, published to the esp32/motor/cmd branch, see Figure 5
Emergency stop works independently and instantly	Outstanding, button wired to the same power command topic as the switch
Disconnecting the ESP32 shows a clear offline status within a few seconds	Implemented via LWT, timing to be measured, see Section 7.6
Flow exported as flows.json and included in the repository	Pending
README updated with screenshots, the topic list and a note on LWT implementation	Pending, topic list available in Section 5 of this report

12. Remaining Work and Next Steps

- Rework the emergency stop so it is more than a second OFF: give the button its own label and payload, force the speed setpoint to 0 alongside the power off, and reset the dashboard switch and slider positions.
- Insert the clamping function node from Section 8.4 into the wire between the speed slider and the MQTT OUT node, since the deployed flow currently publishes the slider value directly.
- Shorten the MQTT keep-alive on the ESP32 with client.setKeepAlive() so the offline indicator appears within a few seconds, then measure the real detection time by pulling power.
- Test the clamping path directly with inject nodes carrying 150, -20 and a non-numeric payload, and confirm in MQTT Explorer that only values between 0 and 100 reach the broker.
- Verify chart persistence across a browser refresh, and set the chart removal window explicitly so the retained history is a documented figure rather than a default.
- Open the dashboard on a phone over the LAN IP and confirm the groups reflow cleanly and the emergency control stays reachable without scrolling.
- Narrow the pressure gauge range from 0 to 1000 down to something like 850 to 950, so the needle carries usable resolution at Amman's ambient pressure.
- Confirm which dashboard package is actually installed and state its name and version in the README, since the palette dialog shows node-red-dashboard installed while the working notes mention the FlowFuse package.
- Disable or delete the four debug nodes, then export flows.json, capture the final screenshots and update the README with the topic table from Section 5 and the LWT explanation from Section 7.

13. References

Node.js downloads, used to install the runtime and npm on Windows. <https://nodejs.org/en/download>

Node-RED, Running Node-RED locally, including the global npm install and the default port 1880.

<https://nodered.org/docs/getting-started/local>

Microsoft, about_Execution_Policies, PowerShell execution policy scopes and the RemoteSigned policy.

https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_execution_policies

node-red-dashboard, Node-RED flow library. Deprecation announcement dated 27 June 2024.

<https://flows.nodered.org/node/node-red-dashboard>

FlowFuse Dashboard, Node-RED Dashboard 2.0 documentation. <https://dashboard.flowfuse.com>

@flowfuse/node-red-dashboard, npm package page. <https://www.npmjs.com/package/@flowfuse/node-red-dashboard>

PubSubClient, Arduino Client for MQTT, API documentation, keep-alive default of 15 seconds and setKeepAlive().

<https://pubsubclient.knolleary.net/api>

PubSubClient repository and README, MQTT 3.1.1 by default, QoS 0 publishing only.

<https://github.com/knolleary/pubsubclient>

HiveMQ, MQTT Essentials Part 9, Last Will and Testament. <https://www.hivemq.com/blog/mqtt-essentials-part-9-last-will-and-testament/>

EMQX, MQTT Will Message explained. <https://www.emqx.com/en/blog/use-of-mqtt-will-message>

Eclipse Mosquitto documentation, mosquitto.conf reference, listener and allow_anonymous directives.

<https://mosquitto.org/man/mosquitto-conf-5.html>

MQTT Explorer. <https://mqtt-explorer.com>